



Instituto Superior Universitario Espíritu Santo

Carrera Desarrollo de software

Desarrollo de Aplicaciones Web

Tema:

SISTEMA DE VETERINARIO

Autor/es:

Hugo Antonio

Romo Espinoza

Genesis Maytte

Solis Alvarez

Profesor:

Andres Loor

Módulo Semestre 2026

1. Problemática

En la actualidad, las clínicas veterinarias no solo se encargan de la salud animal, sino que también gestionan una gran cantidad de información operativa que incluye expedientes clínicos, recetas, exámenes, datos de sus clientes y sus dueños y facturación. A pesar de la existencia de herramientas genéricas de administración, muchas veterinarias locales continúan dependiendo de procesos manuales, hojas de cálculo dispersas o software obsoleto.

Esta falta de centralización genera diversos problemas críticos, comenzando por la pérdida de información debido a que el historial médico de las mascotas suele estar fragmentado, lo que dificulta el seguimiento de tratamientos a largo plazo. Asimismo, se presenta una marcada ineficiencia operativa donde la gestión de citas mediante agendas físicas o chats de mensajería propicia la duplicidad de horarios o el olvido de consultas. A esto se suma la falta de control de los clientes, los exámenes y sus recetas y esto es por la ausencia de un monitoreo. El verdadero reto radica en que construir una solución desde cero implica una inversión alta de tiempo y recursos, donde la seguridad de los datos y la velocidad de respuesta suelen verse comprometidas si no se utilizan los estándares de desarrollo adecuados.

2. Objetivos

Objetivo General

Desarrollar un sistema web integral para la gestión de clínicas veterinarias mediante el uso de un framework de desarrollo, con el fin de optimizar los procesos administrativos, el control de sus pacientes y el seguimiento de expedientes clínicos de manera segura y eficiente.

Objetivos Específicos

El primer paso consiste en diseñar e implementar un módulo de gestión de pacientes y expedientes clínicos que permita a los veterinarios consultar el historial médico de forma rápida y organizada. De igual manera, es necesario desarrollar un componente para el control de inventarios que registre a sus pacientes en base a sus dueños y recetas. Finalmente, el proyecto se enfoca en garantizar la seguridad y escalabilidad de la aplicación aprovechando las herramientas nativas del framework elegido, tales como la autenticación de usuarios y la protección contra vulnerabilidades comunes en la web.

3. Justificación

La realización de este proyecto se justifica desde dos perspectivas fundamentales, siendo la primera el ámbito tecnológico y la segunda el entorno social-empresarial.

Desde el punto de vista técnico, el desarrollo de este sistema demuestra la viabilidad y las ventajas de utilizar frameworks modernos en el ecosistema actual. Estas herramientas provienen con una estructura predefinida que acelera de forma notable el tiempo de desarrollo, garantiza la implementación de buenas prácticas de programación, como la separación de la lógica de negocio

y la interfaz de usuario, y ofrece capas de seguridad robustas para proteger los datos sensibles de los clientes y sus mascotas.

Desde el punto de vista práctico y social, la automatización de una clínica veterinaria se traduce en una mejora directa de la calidad del servicio. Al reducir el tiempo dedicado a tareas administrativas y manuales, el personal médico puede enfocar todos sus esfuerzos en la atención directa de los pacientes. Asimismo, los dueños de las mascotas se benefician de una atención mucho más ágil, profesional y con un menor margen de error en el manejo de sus diagnósticos y tratamientos.

4. Marco Teórico

Para fundamentar el desarrollo de este sistema, es necesario comprender los pilares tecnológicos que sustentan la construcción de aplicaciones web modernas en la actualidad.

5. Tecnologías Utilizadas

Para el desarrollo e implementación de este sistema veterinario, se ha seleccionado un conjunto de herramientas modernas y eficientes que conforman el *stack* tecnológico del proyecto, permitiendo construir una aplicación rápida, ligera y de fácil mantenimiento.

En primer lugar, como núcleo del desarrollo se utiliza Python en su versión 3.9 o superior, un lenguaje de programación ampliamente reconocido por su sintaxis clara y su robustez en el manejo de datos. Sobre este lenguaje se implementa FastAPI, un *framework* de alto rendimiento diseñado específicamente para la construcción de interfaces de programación de aplicaciones (APIs). Esta herramienta se encarga de gestionar de forma automática las tareas críticas de la aplicación web, tales como el enrutamiento de las peticiones de los usuarios, la validación de los datos que ingresan al sistema mediante tipos de datos nativos de Python y la seguridad general de la plataforma.

Para el almacenamiento y la persistencia de la información, se ha optado por SQLite en combinación con SQLAlchemy como mapeador objeto-relacional o ORM. Esta estructura permite interactuar con la base de datos utilizando código de programación puro en lugar de consultas SQL complejas, lo que asegura que los registros de los pacientes, los expedientes clínicos, las citas y el inventario mantengan una relación lógica, sin sufrir duplicidad y garantizando la integridad de los datos de la clínica de manera local y eficiente. Adicionalmente, el sistema requiere la capacidad de generar reportes formales, tales como recetas médicas o resúmenes de historial clínico, por lo que se integra la herramienta WeasyPrint para transformar el código visual directamente en documentos en formato PDF listos para su impresión o descarga.

Por el lado del cliente, es decir, la interfaz con la que interactuará el personal médico y administrativo, se emplean tecnologías estándares como HTML5 para la estructura general de las pantallas y Bootstrap 5 para el diseño visual, lo que permite que el sistema sea completamente adaptable a computadoras, tabletas o teléfonos móviles. La comunicación entre esta interfaz y el servidor se gestiona mediante JavaScript utilizando Fetch API, una tecnología que permite enviar y recibir información en segundo plano sin la necesidad de recargar la página por completo, mejorando drásticamente la experiencia del usuario. Finalmente, para las funciones de

comunicación de la clínica, como el envío de recordatorios de citas o notificaciones, se implementa la librería `aiosmtplib` para el envío asíncrono de correos electrónicos, utilizando la plataforma Mailtrap en la etapa de desarrollo para realizar pruebas controladas de envío en un entorno seguro antes del despliegue real.

6. Arquitectura del Sistema

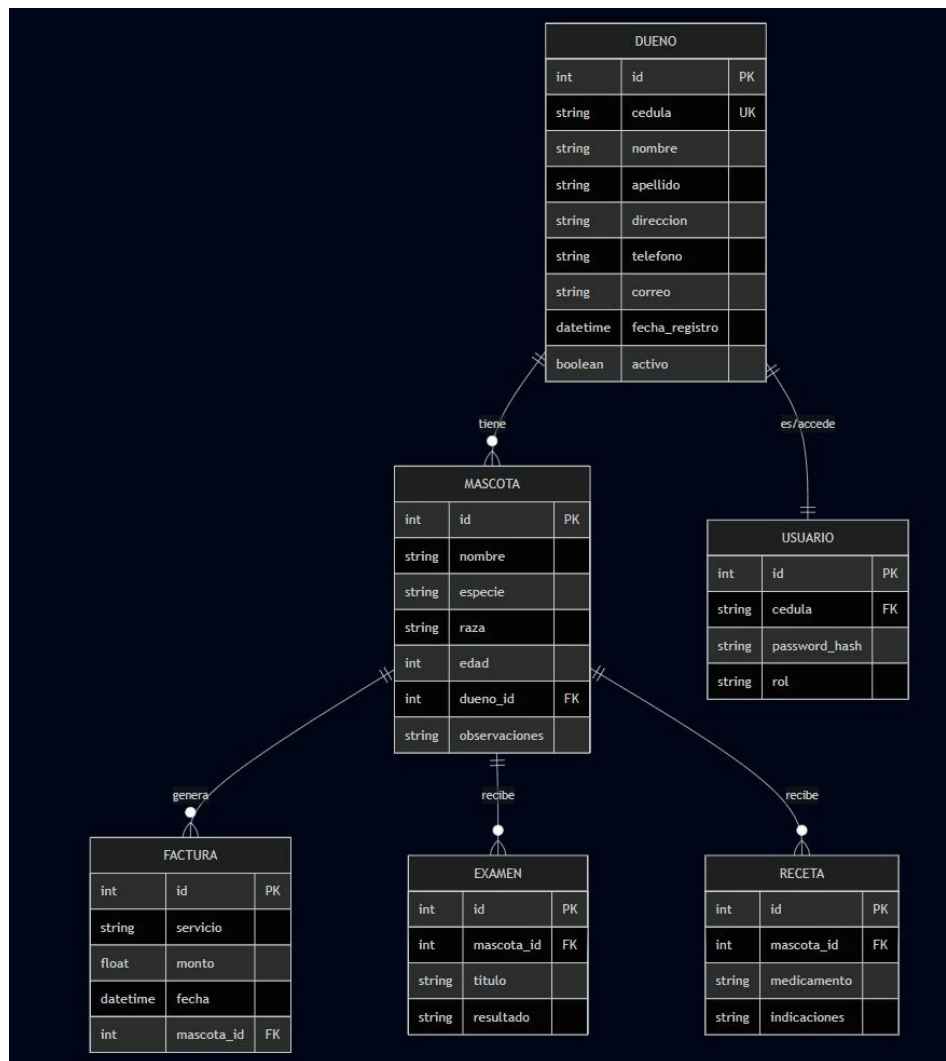
La estructura interna del sistema veterinario se organiza bajo un modelo que prioriza el rendimiento y la disponibilidad de la plataforma, evitando que la aplicación se congele o se vuelva lenta cuando se realizan procesos complejos.

El flujo de información comienza en el lado del cliente, donde el usuario interactúa con la interfaz visual a través de su navegador web. Cada acción que realiza el personal, como agendar una nueva consulta o buscar el expediente de una mascota, se traduce en una petición HTTP que viaja a través de internet con destino al servidor configurado con FastAPI. Una vez que el *framework* recibe esta solicitud, se encarga de procesarla, verificar los permisos del usuario e interactuar con la base de datos relacional a través del ORM para recuperar o actualizar la información que se ha solicitado.

El aspecto más destacado de esta arquitectura es la gestión de las operaciones que demandan un alto consumo de recursos informáticos, como lo son la generación de archivos PDF detallados y el envío masivo de correos electrónicos de notificación. Para evitar que el usuario tenga que esperar en la pantalla mientras estos procesos finalizan, el servidor FastAPI delega de forma inmediata estas actividades pesadas a una cola de tareas en segundo plano conocida como *BackgroundTasks*.

De esta manera, el servidor puede responderle al cliente de manera instantánea confirmando que su solicitud ha sido recibida y procesada con éxito, mientras que, internamente y de forma paralela, el sistema continúa trabajando en la creación del documento o en la conexión con el servidor de correos. Esta separación de responsabilidades asegura que el sistema veterinario se mantenga siempre fluido, rápido y disponible para atender nuevas peticiones en la recepción o en el consultorio.

Base de datos



ENDPOINT

1. Análisis de enviar_correo_directo

Esta función es un **servicio asíncrono** diseñado para enviar correos electrónicos con archivos adjuntos (en este caso, reportes en PDF) utilizando un entorno de pruebas SMTP.

- **Preparación del mensaje (EmailMessage):**
 - Se instancia un objeto de correo y se definen los encabezados básicos (From, To, Subject).
 - `set_content(cuerpo)`: Establece el texto principal del mensaje.
- **Gestión del Adjunto (add_attachment):**

- `pdf_buffer.getvalue()`: Es el punto clave. Extrae los datos binarios del archivo PDF que está en memoria (buffer) para convertirlo en el formato que el protocolo de correo necesita.
- Se definen los parámetros `maintype`, `subtype` (necesarios para que el cliente de correo reconozca el archivo como PDF) y el `filename` que verá el usuario.
- **Envío vía `aiosmtplib`:**
 - `await aiosmtplib.send(...)`: Realiza el envío del correo de forma **no bloqueante**. Esto es fundamental en FastAPI para que tu aplicación no se "congele" mientras espera la respuesta del servidor de correo.
 - **Configuración:** Utiliza los parámetros de conexión de **Mailtrap** (un servidor SMTP de pruebas). El hecho de que `use_tls` y `start_tls` estén en `False` es típico para configuraciones de pruebas locales o ambientes de desarrollo donde no se requiere cifrado estricto.

2 Análisis de la Ruta Principal (/)

Esta función es el **controlador de búsqueda avanzada** de tu sistema veterinario. Su objetivo es permitir al usuario filtrar tanto mascotas como dueños en la página de inicio mediante parámetros opcionales en la URL.

1. Lógica de Parámetros Dinámicos

- Recibe múltiples argumentos opcionales (`Optional[str]`), lo que significa que la ruta funciona igual si no envías nada (trae todo) o si envías uno o varios criterios de filtro (trae resultados específicos).
- `db: Session = Depends(get_db)`: Inyecta la sesión de base de datos para ejecutar las consultas de forma segura.

2. Búsqueda de Mascotas (`query_mascotas`)

- **`join(models.Dueno)`**: Es el punto más importante. Relaciona la tabla de mascotas con la de dueños, permitiendo filtrar mascotas basándose en el nombre o apellido de sus propietarios.
- **`ilike(f"%{valor}%")`**: Es la técnica de búsqueda flexible.
 - `ilike` hace que la búsqueda **no distinga entre mayúsculas y minúsculas** (case-insensitive).
 - Los símbolos `%` actúan como comodines (buscan coincidencias parciales en cualquier parte del texto).
- **Filtros compuestos:** En el caso de `nombre_dueno`, se usa el operador lógico OR (`|`) para buscar el término tanto en el nombre como en el apellido del dueño simultáneamente.

3. Búsqueda de Dueños (`query_duenos`)

- Realiza una consulta independiente enfocada en la tabla `Dueno`.

- **.strip():** Se utiliza para limpiar los espacios en blanco antes de buscar, evitando que una búsqueda falle simplemente porque el usuario presionó la barra espaciadora por error.

4. Respuesta Final

- `templates.TemplateResponse:` Envía al motor de plantillas (Jinja2) dos colecciones de datos: mascotas y dueños, que ya contienen los registros filtrados (o todos, si no se aplicó ningún filtro).

3 Registro de Dueños

Esta función procesa el registro de un nuevo cliente mediante un formulario HTML. Su lógica se divide en tres pasos:

1. **Validaciones de Formato:**
 - Usa `EMAIL_REGEX` (expresión regular) para validar el correo electrónico.
 - Verifica que la **cédula** tenga exactamente 10 dígitos numéricos y el **teléfono** sea solo numérico.
2. **Validación de Integridad:**
 - Consulta la base de datos para asegurar que no exista otro dueño con la misma cédula (evita duplicados).
3. **Gestión de Errores y Registro:**
 - **Si hay error:** Recarga `index.html` enviando un mensaje de alerta (`error_dueno`) al usuario para corregir los datos.
 - **Si es exitoso:** Crea el objeto `models.Dueno` con los datos limpios (`.strip()`), guarda en BD con `db.commit()` y redirecciona a la página principal (/) usando `status_code=303`

4 Registro de Mascotas

Esta función procesa el alta de una nueva mascota, vinculándola directamente a un dueño existente a través de su identificador.

- **Validación de Duplicidad:** Antes de guardar, realiza una consulta compleja filtrando por todas las características (nombre, especie, raza, edad y `dueno_id`) para evitar registros redundantes del mismo animal.
- **Manejo de Errores:** Si detecta una mascota idéntica ya registrada, recarga la página principal (`index.html`) manteniendo el contexto de datos (lista de mascotas y dueños) y muestra un mensaje de advertencia (error).
- **Persistencia:**
 - Si es válida, crea la instancia `models.Mascota`.
 - `db.refresh(nueva_mascota):` Actualiza el objeto con los datos recién generados por la base de datos (como el ID autoincremental).
 - Redirecciona al usuario a / (home) para refrescar la vista.

5 Actualización de Mascota

Esta función permite modificar la información de una mascota existente en la base de datos utilizando el método HTTP PUT.

- **Identificación:** Recibe mascota_id desde la URL para localizar el registro específico.
- **Validación de Existencia:** Antes de procesar, verifica que la mascota exista; si no la encuentra, lanza un HTTPException 404 para informar que el recurso no es válido.
- **Actualización y Persistencia:**
 - Asigna los nuevos valores recibidos del formulario a los atributos del objeto mascota.
 - db.commit(): Guarda los cambios en la base de datos.
 -
 -
 - db.refresh(mascota): Sincroniza el objeto local con el estado actual en la base de datos.

6 Eliminación de Mascota

Esta función gestiona la eliminación definitiva de un registro de mascota utilizando el método HTTP DELETE.

- **Búsqueda y Validación:** Localiza la mascota por su mascota_id. Si no existe en la base de datos, interrumpe el proceso lanzando un HTTPException 404.
- **Acción de Borrado:**
 - db.delete(mascota): Marca el objeto para ser removido de la base de datos.
 - db.commit(): Ejecuta la transacción y confirma la eliminación permanente.

7 Consulta de Exámenes

Esta ruta es la encargada de visualizar el historial clínico de una mascota específica mediante una consulta de relación (tipo "uno a muchos").

- **Validación de Paciente:** Busca a la mascota por su mascota_id. Si no existe, devuelve un error **404** (Paciente no encontrado).
- **Consulta de Relación:** Recupera todos los registros de la tabla Examen que tengan el mascota_id correspondiente.
- **Renderizado de Vista:** Carga la plantilla examenes.html, inyectando tanto los datos de la **mascota** (para mostrar su nombre y detalles) como la **lista de exámenes** asociados, permitiendo que el usuario vea el historial clínico completo del animal en una sola página.

8 Registro de Nuevo Examen

Esta función procesa el guardado de un nuevo examen clínico vinculado a un paciente específico.

- **Captura de Datos:** Recibe el `mascota_id` (desde la URL) y los detalles del examen (titulo, resultado) desde el formulario.
- **Persistencia:** Crea una nueva instancia de `models.Examen`, la agrega a la sesión y ejecuta `db.commit()` para guardar el registro en la base de datos de manera definitiva.
- **Redirección Post-Operación:** Tras el guardado exitoso, realiza un `RedirectResponse` (código **303**) para devolver al usuario a la vista de la lista de exámenes (`/mascota/{mascota_id}/exámenes`), permitiendo ver el examen recién creado inmediatamente.

9 Consulta de Recetas

Esta ruta es el punto de acceso para visualizar el recetario médico de una mascota.

- **Validación de Paciente:** Busca a la mascota por su `mascota_id`. Si no existe, detiene la ejecución con un error **404** (Paciente no encontrado).
- **Renderizado:** Carga la plantilla `recetas.html`, pasando el objeto `mascota` como contexto para que la vista pueda mostrar la información del paciente (nombre, especie, etc.) antes de listar o crear las recetas correspondientes.

10 Registro de Nueva Receta

Esta función es el controlador encargado de persistir una nueva receta médica asociada a un paciente.

- **Captura de Datos:** Recibe el ID de la mascota desde la URL y los detalles del tratamiento (medicamento, indicaciones) a través de un formulario.
- **Persistencia:** Instancia un nuevo objeto `models.Receta`, lo registra en la sesión y confirma la operación con `db.commit()`.
- **Redirección Post-Operación:** Utiliza `RedirectResponse` con código **303** para redirigir al usuario de vuelta a la vista de recetas (`/mascota/{mascota_id}/recetas`), asegurando una navegación fluida tras el guardado.

11 Consulta de Facturas e Historial

Esta ruta permite visualizar todas las transacciones financieras y el historial de cobros asociados a una mascota específica.

- **Validación de Paciente:** Realiza una búsqueda por `mascota_id`. Si la mascota no existe en la base de datos, interrumpe el flujo y devuelve un error **404** (Paciente no encontrado).
- **Consulta de Relación:** Recupera todas las instancias de la tabla `Factura` que coincidan con el `mascota_id` proporcionado.
- **Renderizado de Vista:** Carga la plantilla `facturas.html`, enviando al contexto tanto el objeto `mascota` (para identificar el paciente) como la lista de facturas encontradas, permitiendo listar el historial financiero completo en la interfaz.

12 Registro de Nueva Factura

Esta función es el controlador encargado de registrar un nuevo movimiento financiero asociado a un paciente.

- **Captura de Datos:** Recibe el `mascota_id` desde la URL y los detalles del cobro (servicio, monto) desde el formulario, donde monto es procesado automáticamente como tipo float.
- **Persistencia:** Crea una instancia de `models.Factura`, la añade a la sesión y ejecuta `db.commit()` para guardar el registro en la base de datos.
- **Redirección Post-Operación:** Realiza un `RedirectResponse` con código **303** hacia la ruta `/mascota/{mascota_id}/facturas`, permitiendo que el usuario vea reflejado el nuevo cargo en el historial financiero del paciente inmediatamente.

13 Actualización de Dueño

Esta función implementa la lógica necesaria para editar los datos personales de un cliente, asegurando la integridad de la información.

- **Validación de Existencia:** Busca al dueño por su ID único; si no se encuentra, retorna un error **404**.
- **Limpieza de Datos:** Utiliza `.strip()` en todos los campos de texto para eliminar espacios accidentales al inicio o final.
- **Validaciones de Integridad y Formato:**
 - **Formato:** Verifica (mediante `.isdigit()`, longitud y `EMAIL_REGEX`) que la cédula, teléfono y correo sean válidos.
 - **Unicidad:** Comprueba que la cédula ingresada no esté ya registrada en la base de datos bajo otro ID distinto al que se está editando (evita conflictos de datos).
- **Persistencia:** Si todas las validaciones son correctas, actualiza los campos, ejecuta `db.commit()` y retorna un mensaje de éxito en formato JSON.

14 descargar historial de facturas, Generación de Reportes PDF

Estas rutas permiten exportar el historial clínico o financiero de un paciente a un documento PDF descargable.

- **Lógica Común:**
 - **Consulta:** Busca la mascota por ID y obtiene la lista relacionada (facturas o exámenes).
 - **Preparación del Contexto:** Crea un diccionario con los datos necesarios, incluyendo una marca de tiempo (`current_time`) para que el documento muestre la fecha exacta de emisión.
 - **Generación:** Llama a la función utilitaria `generar_pdf_desde_html`, pasando:
 1. La plantilla HTML específica (`facturas_pdf.html` o `exámenes_pdf.html`).
 2. El contexto con los datos.
 3. El nombre del archivo resultante.

- **Finalidad:** Facilita la entrega de información física o digital al cliente, permitiendo que el sistema genere documentos con formato profesional a partir de los datos almacenados en la base de datos.

15 Descarga de Factura Individual

Esta función permite exportar un comprobante de pago único en formato PDF, a diferencia de la ruta anterior que exporta todo el historial.

- **Identificación Directa:** Consulta la tabla Factura directamente por su factura_id en lugar de hacerlo a través de la mascota.
- **Gestión del Contexto:**
 - Aprovecha la relación definida en SQLAlchemy (factura.mascota) para obtener los datos del paciente automáticamente.
 - Empaqueta la factura en una lista ([factura]) para mantener compatibilidad con la misma plantilla facturas_pdf.html que utilizas para el historial completo.
- **Resultado:** Genera un PDF específico para ese movimiento, facilitando la entrega de recibos individuales a los clientes.

16 Descarga de Examen Individual

Esta función es el equivalente lógico de la descarga de factura individual, pero enfocada en el reporte clínico de un examen específico.

- **Identificación Directa:** Busca el registro en la tabla Examen usando el examen_id. Si no existe, lanza un error **404**.
- **Gestión del Contexto:**
 - Utiliza la relación examen.mascota para obtener los detalles del paciente sin realizar una consulta adicional a la tabla de mascotas.
 - Envuelve el objeto examen en una lista ([examen]) para que la plantilla examenes_pdf.html pueda iterar sobre él correctamente, manteniendo la misma lógica que usas para exportar historiales completos.
- **Finalidad:** Permite generar un documento PDF profesional para un examen específico, ideal para imprimir o enviar por correo a un cliente.

17 Descarga de Receta Individual

Esta función sigue el mismo patrón de diseño que las descargas de facturas y exámenes, permitiendo exportar una receta específica en formato PDF.

- **Identificación Directa:** Consulta la tabla Receta mediante su receta_id. Si no se encuentra, retorna un error **404**.
- **Gestión del Contexto:**
 - Extrae los datos del paciente a través de la relación receta.mascota.

- Envuelve la receta en una lista ([receta]) para que la plantilla recetas_pdf.html pueda procesarla correctamente como una colección de elementos.
- **Finalidad:** Facilita la generación de documentos médicos individuales (recetas con medicamentos e indicaciones) listos para ser entregados al dueño de la mascota.

18 Envío de PDF por Correo

Esta función es un servicio asíncrono avanzado que automatiza la entrega de reportes mediante email.

- **Validación de Integridad:** Comprueba no solo que la mascota exista, sino que tenga un dueño asignado y que este cuente con un correo electrónico registrado. Si falta alguno, lanza un error **400** (Bad Request).
- **Generación de PDF en Memoria:** Utiliza generar_pdf_buffer para crear el documento en la memoria RAM en lugar de guardarlo en el disco, optimizando el rendimiento.
- **Tareas en Segundo Plano (BackgroundTasks):**
 - Este es el punto clave: en lugar de esperar a que el servidor de correo termine de enviar el mensaje, FastAPI libera la respuesta al usuario inmediatamente mientras el proceso de envío se ejecuta de forma independiente (background_tasks.add_task).
 - Esto evita que el usuario experimente una espera prolongada en su navegador mientras el correo es enviado.
- **Respuesta:** Retorna una confirmación inmediata al cliente, confirmando que la tarea de envío ha sido programada correctamente.

19 Descarga de Historial de Exámenes (PDF)

Esta ruta permite al veterinario exportar todo el historial clínico de exámenes de una mascota específica en un archivo PDF.

- **Validación de Paciente:** Realiza una búsqueda por mascota_id. Si no se encuentra al paciente, devuelve un error **404** (Paciente no encontrado).
- **Recopilación de Datos:** Consulta la tabla Examen para traer **todos** los registros asociados a ese mascota_id.
- **Contexto y Generación:**
 - Prepara un diccionario con los datos del paciente, la lista completa de exámenes y la fecha actual (current_time).
 - Invoca a generar_pdf_desde_html apuntando a la plantilla examenes_pdf.html.
- **Salida:** Genera un archivo con un nombre dinámico (historial_examenes_{mascota_id}.pdf) que contiene el resumen clínico completo del paciente, listo para impresión o archivo.

20 Descarga de Historial de Recetas (PDF)

Esta función genera un reporte médico formal del historial completo de medicamentos de un paciente.

- **Validación:** Verifica que la mascota exista; si no, retorna error **404**.
- **Consulta:** Recupera todas las recetas asociadas al ID de la mascota desde la base de datos.
- **Contexto:** Agrupa los datos del paciente, la lista de recetas y la fecha actual para el reporte.
- **Generación:** Utiliza `generar_pdf_desde_html` para convertir la plantilla `recetas_pdf.html` en un archivo PDF descargable con el nombre del paciente.

Capturas del sistemas

Sistema de Gestión Veterinaria

Control Clínico, Propietarios y Facturación Integrada

Gestión de Pacientes

Gestión de Dueños / Clientes

Registrar Nuevo Paciente

Nombre Mascota

Especie

Raza

Edad (Años)

Seleccionar Dueño

Ej: Lulu

Elegir...

Otro

0

-- Selecciona Propietario --

Guardar Mascota

Filtrar por Paciente

Filtrar por Dueño

Especie

Raza

Filtrar Pacientes

Historial de Pacientes

Nombre	Especie	Raza	Edad	Dueño Responsable	Módulos Fichas	Acciones
lulu	Lero	Amazonas	12 años	Hugo romo	Exámenes Recetas Facturas	
lulu	Pero	Poodle	11 años	Genesis Solis de Romo	Exámenes Recetas Facturas	
toby	Tortuga	Galápagos	50 años	Karla Alvarez de Solis	Exámenes Recetas Facturas	

Control Clínico, Propietarios y Facturación Integrada

Gestión de Pacientes

Gestión de Dueños / Clientes

Registrar Nuevo Dueño (Cliente)

Cédula / Identificación

Nombres

Apellidos

Teléfono / Celular

Ej: 0999999999

Ej: Juan

Ej: Pérez

Ej: 0987654321

Correo Electrónico

Dirección Domiciliaria

nombre@correo.com

Ej: Vía a la Costa, Guayaquil

Guardar Dueño

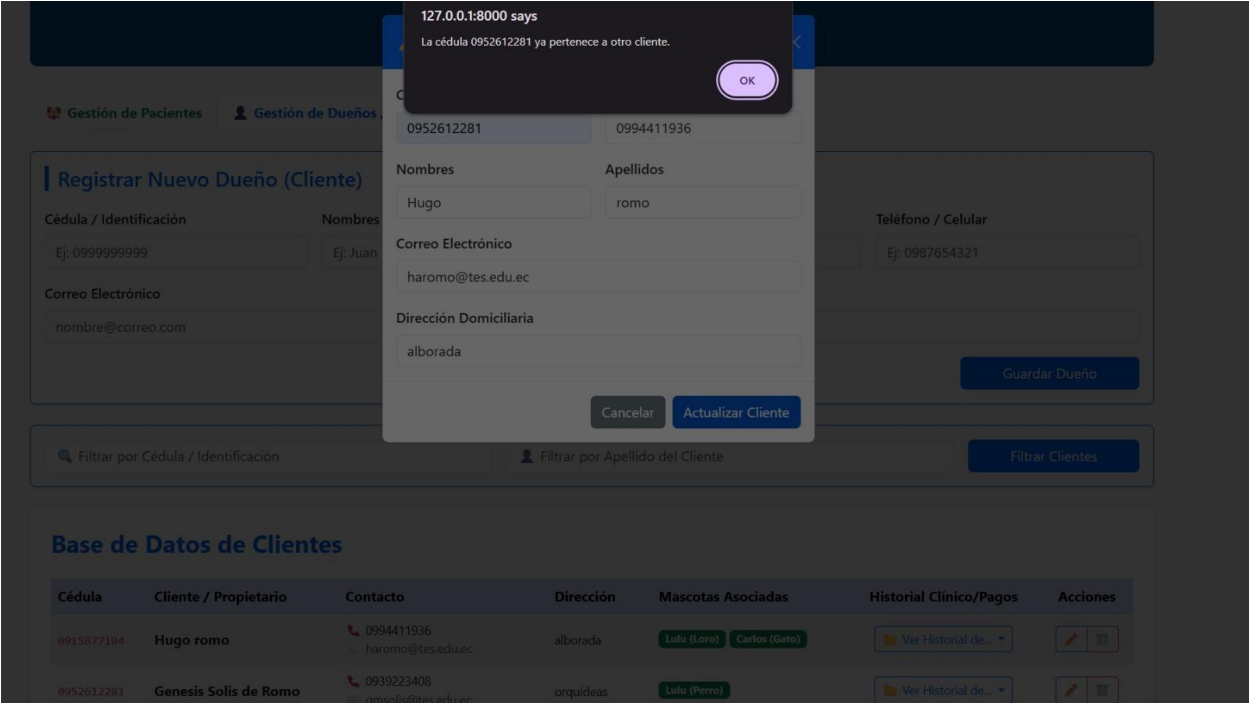
Filtrar por Cédula / Identificación

Filtrar por Apellido del Cliente

Filtrar Clientes

Base de Datos de Clientes

Cédula	Cliente / Propietario	Contacto	Dirección	Mascotas Asociadas	Acciones
0915877104	Hugo romo	0994411936 haromoi@tes.edu.ec	alborada	Lulu (Lero) Carlos (Gato)	Facturas lulu Ficha Médica lulu Facturas carlos Ficha Médica carlos Ver Historial de... Ver Historial de...
0952612281	Genesis Solis de Romo	0939223408 gmsolis@tes.edu.ec	orquideas	Lulu (Perra)	Ver Historial de... Ver Historial de...
		1234567890			



Registrar Nuevo Dueño (Cliente)

Cédula / Identificación

Ej: 0999999999

Nombres

Ej: Juan

Apellidos

Ej: Pérez

Teléfono / Celular

Ej: 0987654321

Correo Electrónico

! Please fill out this field.
s

Dirección Domiciliaria

Ej: Vía a la Costa, Guayaquil

Guardar Dueño

POST	/mascota/{mascota_id}/factura_nueva	Crear Factura	▼
PUT	/dueno/{id}	Actualizar Dueño	▼
GET	/mascota/{mascota_id}/facturas/pdf	Descargar Facturas Pdf	▼
GET	/factura/{factura_id}/pdf	Descargar Factura Individual Pdf	▼
GET	/examen/{examen_id}/pdf	Descargar Examen Individual Pdf	▼
GET	/receta/{receta_id}/pdf	Descargar Receta Individual Pdf	▼
POST	/enviar-pdf-correo/{mascota_id}	Enviar Pdf Correo	▼
GET	/mascota/{mascota_id}/exámenes/pdf	Descargar Exámenes Pdf	▼
GET	/mascota/{mascota_id}/recetas/pdf	Descargar Recetas Pdf	▼
POST	/factura/{factura_id}/eliminar	Eliminar Factura	▼
POST	/receta/{receta_id}/eliminar	Eliminar Receta	▼
POST	/examen/{examen_id}/eliminar	Eliminar Examen	▼
DELETE	/dueno/{dueno_id}	Eliminar Dueño	▼

default

GET	/	Index	⌵
POST	/registrar_dueno/	Crear Dueno	⌵
POST	/registrar/	Crear Mascota	⌵
PUT	/mascota/{mascota_id}	Actualizar Mascota	⌵
DELETE	/mascota/{mascota_id}	Eliminar Mascota	⌵
GET	/mascota/{mascota_id}/exámenes	Ver Exámenes	⌵
POST	/mascota/{mascota_id}/examen_nuevo	Registrar Examen	⌵
GET	/mascota/{mascota_id}/recetas	Ver Recetas	⌵
POST	/mascota/{mascota_id}/receta_nueva	Registrar Receta	⌵
GET	/mascota/{mascota_id}/facturas	Historial Facturas	⌵

Editar Datos del Cliente



Cédula / Identificación

0915877104

Teléfono

0994411936

Nombres

Hugo

Apellidos

romo

Correo Electrónico

haromo@tes.edu.ec

Dirección Domiciliaria

alborada

Cancelar

Actualizar Cliente

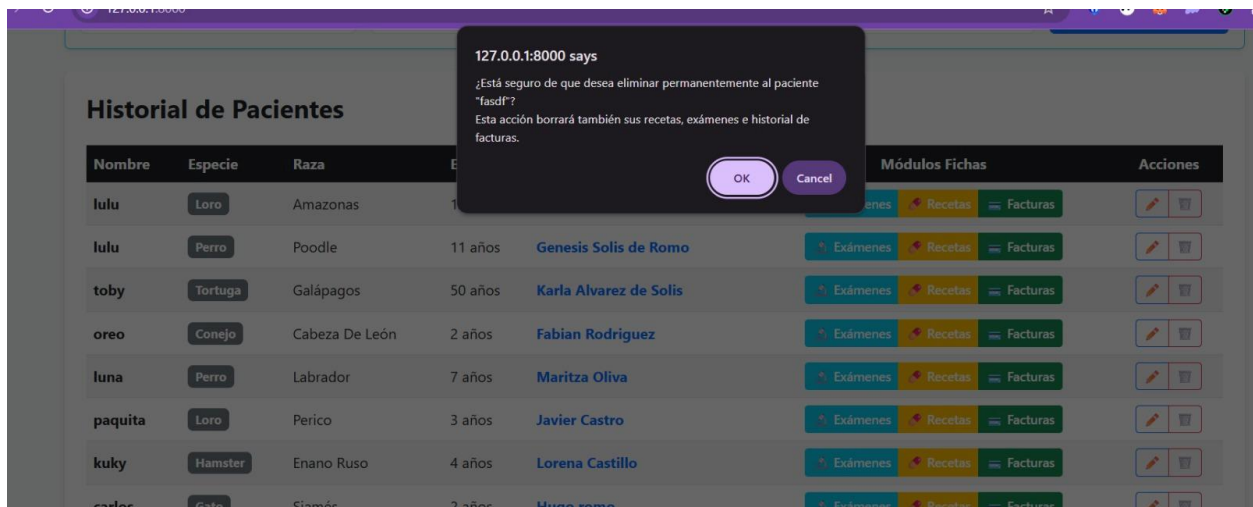
127.0.0.1:8000 says

¿Está seguro de que desea eliminar al cliente "Hugo romo"?

⚠️ ¡ATENCIÓN!: Solo podrá eliminarlo si este propietario ya no tiene ninguna mascota vinculada.

OK

Cancel



Editar mascota

The screenshot shows a form titled "Editar Datos del Paciente" with a close button (X). The form contains the following fields:

- Nombre de la Mascota:** A text input field containing "fasdf".
- Especie:** A dropdown menu with "Tortuga" selected.
- Raza:** A dropdown menu with "De Agua" selected.
- Edad (Años):** A text input field containing "3".
- Dueño Responsable:** A dropdown menu with "Genesis Solis de Romo" selected.
- Observaciones Clínicas:** A text area containing "Sin observaciones".

At the bottom of the form are two buttons: "Cancelar" (grey) and "Actualizar Cambios" (blue).

Panel de exámenes

Volver al Inicio

 **Historial de Exámenes: lulu**

Resultados y diagnósticos del paciente

Descargar Historial Completo en PDF

Subir Nuevo Reporte de Examen

Tipo de Examen (Título)

Ej: Hemograma completo

Resultado del Reporte

Escriba aquí el informe médico...

Guardar Reporte

Reportes Registrados

Título	Fecha	Resultado	Acciones
Hemograma completo	N/A	lulu esta bien	Descargar
otro examen	N/A	dfsdfd	Descargar

Volver al Inicio

Historial de Recetas: lulu

Control de medicamentos y prescripciones médicas

Descargar Historial Completo en PDF

+ Emitir Nueva Receta

Medicamento

Ej: Amoxicilina 500mg

Indicaciones

Ej: Administrar 1 tableta cada 12 horas por 7 días...

Guardar Receta

Recetas Registradas

Medicamento	Indicaciones	Acciones
Paseo diario	15 min 2 veces al día	Descargar
amoxicilina 500	1 cada 12 horas por 7 días	Descargar

Envío de correo

127.0.0.1:8000 says

¿Deseas enviar el historial de facturas al correo del dueño?

OK

Cancel

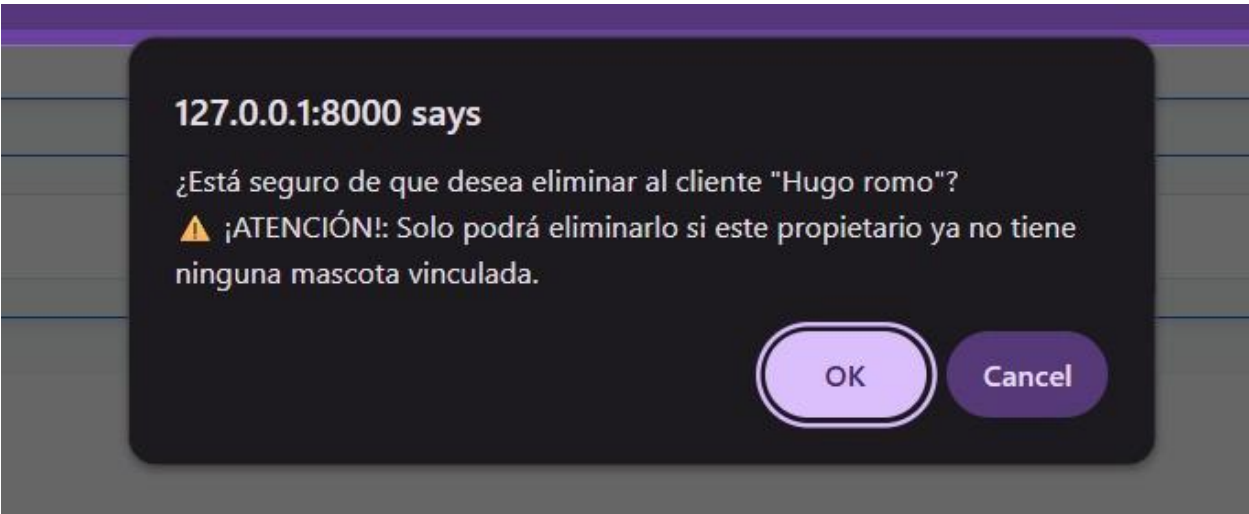
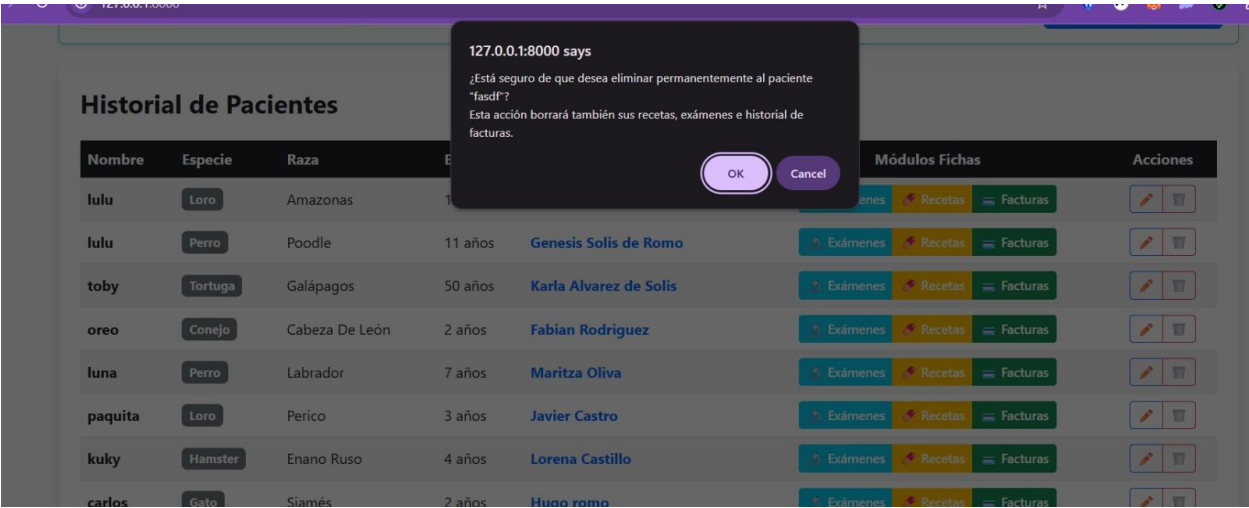
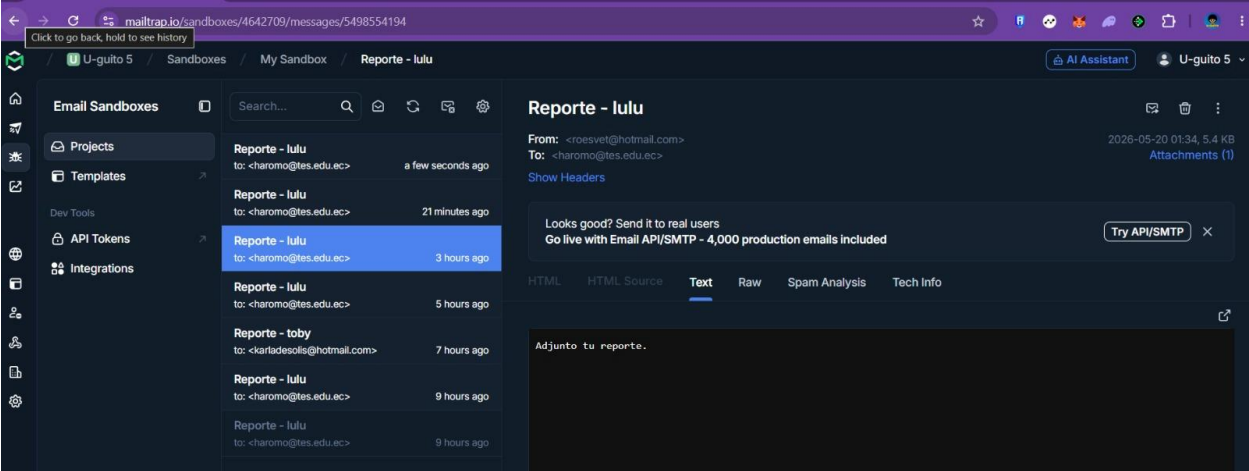
Control de cobros y servicios de la mascota

Descargar Historial Completo en PDF

Registro de Facturas Emitidas

Fecha	Servicio / Concepto	Monto	Dueño Autorizado	Acciones
2026-05-19 10:21	consulta	\$20.00	Hugo romo	
2026-05-19 14:57	dfsdf	\$32.00	Hugo romo	
2026-05-19 20:34	gfdg	\$23.00	Hugo romo	

Verificacion de correo recibido



7. Conclusión

A lo largo del desarrollo de este proyecto, se ha evidenciado que el diseño e implementación de un sistema web integral para una clínica veterinaria no solo resuelve una necesidad administrativa real, sino que también sirve como un escenario ideal para poner a prueba las ventajas competitivas que ofrecen los entornos de desarrollo modernos. La problemática inicial, marcada por la fragmentación de la información, el descontrol en los inventarios y la ineficiencia en el agendamiento de citas, encontró una solución sólida mediante la centralización de datos y la automatización de procesos. Al adoptar un enfoque estructurado, se logró demostrar que es posible transformar la gestión operativa de un negocio de salud animal en una plataforma digital intuitiva, segura y accesible para todo el personal clínico.

Desde la perspectiva del aprendizaje técnico en la materia, la utilización de un *framework* de alto rendimiento como FastAPI, en combinación con el lenguaje Python, resultó ser un factor determinante para el éxito del sistema. Estas herramientas permitieron abstraer la complejidad del código base y concentrar los esfuerzos en las reglas de negocio específicas de la veterinaria. Asimismo, la integración de una arquitectura basada en tareas en segundo plano (*BackgroundTasks*) para procesos pesados como la generación de reportes en PDF con WeasyPrint y el envío de correos con aiosmtplib, aportó un valor técnico significativo. Este enfoque arquitectónico demostró que las aplicaciones modernas no solo deben ser funcionales, sino también eficientes y capaces de ofrecer una experiencia de usuario fluida sin comprometer la estabilidad del servidor.

Finalmente, este proyecto reafirma la importancia que tiene para un estudiante universitario el desarrollo de software basado en buenas prácticas, estándares web actuales y el uso de un *stack* tecnológico equilibrado. La combinación de una interfaz responsiva construida con Bootstrap 5 y un backend asíncrono y relacional con SQLAlchemy y SQLite deja las bases sentadas para que el sistema sea escalable en el futuro. En conclusión, el desarrollo del sistema veterinario no solo cumple con los objetivos académicos planteados para la materia, sino que también representa una valiosa experiencia práctica sobre cómo la tecnología y los *frameworks* actuales pueden optimizar y profesionalizar los servicios de cualquier organización en el mundo real.